Alex Castillo González

Applied AI Engineer · LLM integration: streaming, tool calls, usage, migrations

Santo Domingo, Dominican Republic — UTC−4 · Independent contractor (Spanish passport) · Spanish (native), English (professional) alex.castillog33@gmail.com · github.com/pcbeingused333 · linkedin.com/in/alexcastillogonzalez · portfolio-alexgonzalez33.vercel.app

SUMMARY

Applied AI engineer working in Python on retrieval and agent systems, and on the layer that decides whether they survive real users: evaluation, failure handling, and knowing which numbers actually moved. My main retrieval project answers over the text of the **GDPR** and is built around a constraint that regulated domains impose and generic RAG ignores — every statement has to name the provision it came from, and the system has to decline when the source does not cover the question. Both projects ship with the harness that measures them — retrieval, citation accuracy and abstention for the RAG system, tool trajectories and answer grounding for the MCP agent — and in each case the harness found defects the tests did not. That measurement layer is now a published library, `ragcite`, which asserts that it reproduces the numbers of the project it was extracted from. Fullstack background across Python, TypeScript and Ruby, with production experience shipping and operating what I build. Most of my recent work is the **integration layer between applications and LLM providers** — streamed tool calls, token and cost accounting, and code that breaks when a provider changes or retires a model — fixed upstream in the SDKs and frameworks themselves: **160 merged pull requests** across 64 projects since July (83% of those decided were merged; median 16 hours to merge), including 29 in **LangChain.rb**, 12 in deepset's **Haystack** and its integrations, and fixes in **LiteLLM**, **pydantic-ai**, **Arize Phoenix**, **Opik**, **redis-py** and **OpenTelemetry**'s GenAI instrumentation.

SKILLS

AI / LLM — Python · LLM APIs (Groq, OpenAI-compatible) · LangChain · LangGraph · ReAct agents and tool use · **Model Context Protocol (MCP)**: building servers and clients · **agent trajectory evaluation** (tool selection, ordering, argument accuracy, answer grounding) · RAG: chunking strategy, retrieval tuning, **structure-aware citation at the provision level** · **LLM evaluation**: known-answer datasets, retrieval metrics (hit@1, recall@k, MRR), **abstention and unsupported-claim rate**, LLM-as-judge for faithfulness / relevancy / correctness / context precision · cross-lingual retrieval · resilience to unreliable model output (retry with rate-limit backoff, graceful degradation)

Regulatory / legal text — parsing legislative sources into citable provisions (EUR-Lex / Official Journal markup) · citation integrity as a design constraint · evaluating refusal on out-of-corpus questions · GDPR structure (data subject rights, controller and processor obligations, breach notification, administrative fines)

Data & retrieval — embeddings (`sentence-transformers`, BGE, MiniLM) · FAISS · PostgreSQL + `pgvector` · PDF ingestion pipelines · structured corpus construction

Backend — Python · pytest · Ruby on Rails · PostgreSQL · REST APIs · JSON/API integrations (Jira API)

Web — TypeScript · Next.js · React · Tailwind · JavaScript · HTML/CSS/SCSS

Infra & tooling — **AWS** (Lambda container images, DynamoDB, ECR, IAM least-privilege, CloudWatch, Budgets) · **Terraform** · **CI/CD with GitHub Actions, OIDC federation** · Docker · Docker Compose · Vercel · Streamlit Community Cloud · Git (GitHub, GitLab) · AI-assisted development (Claude Code, Cursor)

EXPERIENCE

Churrería Calderón — Founder & owner-operator · Toronto, Canada

Sep 2025 – Jul 2026 *(business closed July 2026)*

- Moved my churro business from Spain to Toronto on my own: signed a lease in Kensington Market, imported the production machinery, registered the business and obtained every permit needed to operate legally.

- Fitted out the store from my arrival in September 2025 and opened on 16 December, then ran it with no hires — production, service, customers, purchasing, cash and compliance — until it closed in July 2026.

Family churrería business — Manager · Catalonia, Spain

2023 – Aug 2025 · mainly summer seasons *(and in the same business before that)*

- Managed the family business, mainly through the summer seasons: production, service, customers, purchasing and cash.
- Took it from a mobile trailer to fixed premises, which meant rebuilding the operation around a different site, different hours and a different customer base.
- Also worked my father's stand on the fairground circuit.
- Years of reading what customers actually ask for, in person, hundreds of times a day. That is the same instinct a support-facing product feature needs, and it is why the AI widget I later built was scoped to the four questions people really ask.

Le Wagon — Part-time Programming Teacher · Remote (France)

Oct 2022

- Taught on the new part-time flex cohort of the fullstack bootcamp: Ruby, object-oriented programming, SQL and PostgreSQL, HTML/CSS/JavaScript, and building on Ruby on Rails.
- Supported students through exercises and debugging during live sessions.

TECNOBIT (Grupo Oesía) — Fullstack Developer · Valdepeñas, Spain

Aug 2022 – Nov 2022 · On-site

- Shipped features into a long-running internal application maintained by a team of four engineers and a systems engineer.
- **Backend:** wrote Ruby routines that pulled data from JSON files and the **Jira API**, transformed it and persisted it to PostgreSQL; implemented multi-stage validation processes that gated document generation and downloads.
- **Frontend:** built form-driven pages and PostgreSQL-backed views with role-dependent data (users and managers saw different data), and surfaced the state of backend subprocesses so users could follow a document download to completion.
- Worked across a codebase distributed over both GitHub and GitLab.

SELECTED PROJECTS

Business Ops Agent — MCP server + agent, with trajectory evaluation

Live demo · Code

A **Model Context Protocol server** exposing a business's operations (catalog, booking capacity, stock, quoting, orders) as tools any MCP client can call — Claude Desktop, Cursor, or the LangGraph agent bundled with it. The agent carries no business rules: tools are discovered at runtime, so adding one requires no agent change.

- Built an evaluation harness that scores **tool trajectories**, not just answers: which tools were called, in what order, with which arguments, and whether every figure in the reply traces back to a tool result. The grounding check needs no judge model and is therefore deterministic and free — it holds even when a fabricated number happens to be correct.
- The harness caught the agent answering "I don't have that information" with zero tool calls, and caught it **intermittently**, which single-run testing misses; scenarios can be repeated to measure flaky behaviour. Scored 11/12, mean 0.98.
- Chose not to use `langchain-mcp-adapters`: it pins `mcp<2` and would have forced a working server back to an older SDK. Wrote a 60-line bridge instead, passing the server's JSON Schema straight through so no tool signature is duplicated.
- **Deployed it to AWS** as a remote MCP server — Lambda container behind a Function URL, DynamoDB single-table store, least-privilege IAM, all in Terraform. Storage sits behind an interface, so the same server runs on SQLite locally and DynamoDB in production with no change above the backend.

- CI/CD on every push to main via GitHub Actions, authenticating with **OIDC** rather than a stored access key, and scoped to one branch so a fork's pull request cannot assume the deploy role. The role deliberately cannot apply infrastructure: it can ship an image and repoint the function, nothing more.
- Python, MCP 2.0, LangGraph, Groq, AWS (Lambda, DynamoDB, ECR, IAM), Terraform, Docker, GitHub Actions, pytest (161 tests).

Ask the GDPR — retrieval over regulation, with citations that can be checked

Live demo · Code

LangGraph agent over the full text of the GDPR. Every answer names the provision behind it — `Art. 33(1)`, not a page number — and the system declines when the source does not cover the question. Two modes behind one flag: an in-memory FAISS demo on a free 1 GB container, and a pgvector-backed production path.

- **Made the citation structural rather than incidental.** A regulation is cited by article and paragraph; the page a provision lands on is an artefact of typesetting, and a reader sent to "page 14" can confirm nothing. So the corpus is not a PDF: a builder parses the Official Journal text from EUR-Lex (not a mirror — in a system whose claim is checkable citations, the text has to come from the authority the citation names) into **414 provisions** carrying article, paragraph, title and chapter as metadata. The chunk size was then chosen so that **97% of provisions survive as exactly one chunk**, because a chunk straddling Art. 33(1) and 33(2) gets attributed to one of them and cites the wrong paragraph.
- **Built an eval for the answers that should never be given.** In legal text a retrieval miss announces itself; an invention is fluent, confident and indistinguishable from a correct answer, and a citation the model reasoned its way to rather than read makes it more convincing. So the harness scores refusal against questions the Regulation does not answer but every model has read about — adequacy decisions by country, the text of the standard contractual clauses, Schrems II, a CCPA penalty — alongside a deterministic check for citations that appear in the answer but never in the retrieved passages. Every question scored so far declined correctly and specifically, naming the provision that creates the mechanism and stating that the detail asked for is not in the text; none produced an unsupported answer or a citation that was never retrieved.
- Changed the ground truth from matching text to **matching the cited provision**, which is stricter (retrieving the right words is not retrieving the right authority) and removes the chunk-boundary false misses the substring approach suffered.
- **Re-ran every measurement when the corpus changed, and one result reversed.** Embedding the article heading measurably *hurt* retrieval under the old embedding model and measurably *helped* under the new one; carrying the first conclusion forward would have shipped the worse setting on the strength of real evidence. The model swap itself was worth 8/20 → 13/20 at rank 1 for 42 MB of RAM, against a hard 1 GB ceiling measured with the index loaded, not just the model.
- Three parse bugs caught by sanity checks that now block the build: the closing article silently absorbing the signatures and all 21 footnotes, nested sub-points truncated by a non-greedy regex that could not handle nested markup, and the 26 definitions of Art. 4 collapsing into one uncitable record.
- Fixed cross-lingual retrieval (Spanish 0/4 at rank 1 against an English index) by translating the retrieval query rather than paying ~350 MB for a multilingual embedding model that did not fit the 1 GB budget — a measured trade-off.
- CI on every push runs the suite headlessly, including a boot test of the app itself — the host redeploys straight from `main`, so the suite is the only gate before the public demo. One test forces the embedding loader to raise and asserts the first render still succeeds, proving nothing heavy sits on the render path.
- Python, LangChain, LangGraph, Groq, FAISS, pgvector, Streamlit, Docker, GitHub Actions, pytest (72 tests).

ragcite — the evaluation layer of the two projects above, as a library

Code

Retrieval metrics, citation grounding, abstention scoring and tool-trajectory scoring, extracted from the two harnesses above and generalized. No LangChain, vector store or LLM client is imported: you bring the retriever and the judge, `ragcite` scores what they return.

- **It reproduces the numbers of the project it came from, and asserts it.** The dogfood example scores one FAISS index twice — once with `ragcite`, once with the original project's own independent metrics code — and exits non-zero if they disagree. They match exactly (hit@1 13/25, recall@k 17/25, MRR 0.59), which is what makes "extracted from a production harness" a check rather than a claim.

- Retrieval and grounding need no model at all: `hit@1` / `recall@k` / `MRR` are id matching and grounding is a set membership test, so both are deterministic and free to run on every commit. A fabricated citation is caught however fluently it reads.
- `--min-hit-at-1`, `--min-recall`, `--min-mrr` turn a run into a CI gate that exits 2 on a regression. Without a threshold flag the command always exits 0 — a report, not a gate, and it stays that way by default.
- `check_judge_independence` refuses to run when the judge and the system under test are the same model. That defect is why it exists: my own harness spent weeks marking a model as its own examiner.
- Zero runtime dependencies, MIT, 155 tests, CI on Python 3.9–3.12.

AI Website Chat Widget — embeddable business assistant

Live demo · Code

Drop-in chat widget for small-business sites, grounded strictly in the business's own content. Reusable for any client from a single config file. Next.js, React, TypeScript, Tailwind, Groq, Vercel.

Semantic Recommender — embedding-based recommendations

Code

Recommendation engine built on vector embeddings with a feedback loop that refines results over time, as a reusable backend for platforms that have outgrown rule-based filters. Python, embeddings, pgvector, PostgreSQL.

OPEN SOURCE

160 merged pull requests across 64 open-source projects since July 2026, 83% of those decided (32 closed unmerged); median 16 hours to merge, 94 merged within 24 hours. Most are in the layer between applications and LLM providers. Each ships a regression test that fails without the fix. Five that show the kind of work:

- **LiteLLM — streamed calls billed differently from non-streamed ones** (#44795). *Bug:* a streamed Vertex AI pass-through call cost less than the same call non-streamed. *Cause:* the stream builder priced the call without its location, so regional endpoints missed their 10% uplift and custom-priced deployments fell back to list price. *Fix:* the logger prices both modes, including cut-off streams, with the deployment's location. *Impact:* spend tracking in a 61k★ proxy now matches across streaming and non-streaming.
- **LLM Tornado — three parallel tool calls executed six times** (#205). *Bug:* with Anthropic streaming and `Microsoft.Extensions.AI`, each parallel tool ran more than once and the follow-up request was rejected. *Cause:* every `content_block_stop` re-yielded all tool calls seen so far (1, then 1+2, then 1+2+3), and the tool-invocation middleware acted on each. *Fix:* yield the calls once, at the end of the stream, with the final usage. *Impact:* merged the same day; seven merged in that repo, including the streamed input and cache token counts (#214).
- **Haystack — async paths that drifted from the sync ones** (#12662, integrations #3790). *Bug:* 50 expanded queries meant 50 concurrent embedding calls whatever the limit; an `asyncio.Lock` was shared across event loops. *Cause:* `run_async` re-implemented `run` without its guards, and the tests only called `run`. *Fix:* restore the limits and per-loop locks, with async tests. *Impact:* 12 merged across deepset's framework; written up.
- **LangChain.rb — the provider and tool-calling layer** (29 merged). *Bugs:* `tool_choice` sent in the wrong shape per API, an agent loop that could never stop calling a forced tool, streamed tool calls lost on Ollama, thinking blocks dropped from the tool-use loop. *Fix:* one adapter at a time across Anthropic, OpenAI, Gemini, Ollama, Cohere and Mistral. *Impact:* most merged the same day; it is now most of that layer.
- **Model retirements — defaults that stop working on a date** (AgentScope-Java #3670, go-micro #5011). *Bug:* SDKs defaulting to Claude models that Anthropic retires (or already had). *Cause:* hard-coded model IDs, and in AgentScope a context-window table with no Claude 5 entry, so the obvious one-line swap would have resolved the window to 0. *Fix:* new default plus the missing table entry, with a test that fails on the old one. *Impact:* merged the same day in both.

Beyond those case studies, the work falls into five patterns:

- **When the LLM provider changes, the code built on it breaks.** Claude 5 and the 1.x Anthropic SDK broke the same few assumptions in about a dozen projects: reading a reply as `content[0].text` when the first block is now a `thinking` block; sending `temperature` or a forced `tool_choice` that current models reject with a 400; calling the removed Text Completions API; and model catalogs that didn't recognise new or Bedrock model IDs. Merged in hostedgpt, reward-bench, TruLens, chatlas, WeKnora and LangChain.rb, with a median of about eight hours to merge, because users were hitting them that week. Examples: `Tencent/WeKnora` #4155 — internal pipelines (query understanding, entity extraction, compaction) got a 400 on six Claude models catalogued as accepting sampling parameters; `AllYourBot/hostedgpt` #823 — auto-titling read the first content block and got `nil`, intermittently, because adaptive thinking skips short prompts.
- **Async paths in Haystack that drifted from the sync ones.** `run_async` implementations that ran blocking work on the event loop, held an `asyncio.Lock` across event loops, ignored `max_workers`, skipped batching, or re-wrapped errors the sync path re-raises. These bugs hide from test suites that only call `run`. Eight merged, plus four `to_dict` fixes from an AST audit of `__init__` against serialization, where a setting silently reverted when a pipeline was reloaded. Examples: `haystack-core-integrations` #3790 (the lock, reported first as #3789), `haystack` #12662 (50 expanded queries meant 50 concurrent embedding calls whatever the limit). Written up as Four concurrency bugs on Haystack's async path.
- **The provider and tool-calling layer of LangChain.rb.** 29 merged, usually the same day, across its Anthropic, OpenAI, Gemini, Ollama, Cohere and Mistral adapters: `tool_choice` `"none"`/`"auto"` sent in the shape each API expects, an agent loop that could never stop calling a forced tool, streamed tool calls lost on Ollama, thinking blocks dropped from the tool-use loop, a migration to the cohere-ruby 1.0 (v2) API, and tool errors returned to the LLM instead of ending the run. Examples: #1063, #1058; all of them.
- **One rare but legitimate input that brings down the whole flow.** An empty `choices` list, a `nil` body, a division by zero, a bare `%` in an email layout — often inside a `rescue`/`except` written to catch a different exception. Merged across LangChain.rb and three Ruby gems: `courrier` (11 merged, including three new providers and cc/bcc that six providers silently dropped), `perron` (4) and `icons` (2); open in instructor, openinference and guardrails.
- **Evaluation metrics that return a wrong number without failing.** Retrieval metrics above 1.0 on rankings with repeated ids, pass@k averaged as pass/fail, a Spearman correlation computed on rankings with duplicates, an eval case that could never fail. Merged in `opik` #8276, `phoenix` #16002 and `pydantic-ai` #7936; five open in `llama-index-core` and four in `deepeval`. `pydantic-ai` #7927, a judge grading binary output rendered as one decimal byte per line, was reported with a reproduction and accepted.

Also merged: twelve in the Rails app `we-promise/sure` (personal finance, 10k★: provider-setup flows from Add Account, wrong HTTP methods, data kept when merging duplicates); nine in `pipenv` and one in `pdm` (PEP 751 `pylock.toml` and inline script metadata); token usage of failed Responses API calls in `pipecat`; Ruby and Rails tooling — two in `rubocop-rspec` and a drop-down ordering fix in `human-essentials` — and two in `pyfenn/fenn`. Every pull request, open and closed.

What worked: small PRs with a failing-first regression test and the local verification pasted in; answering reviews quickly; going back to maintainers who already trusted the work (LangChain.rb went from 6 to 29 merged); and fixing what a release had just broken. **What didn't:** crowded repos where the maintainer fixes it first, and repos where CI never runs for fork PRs.

EDUCATION

Le Wagon — Fullstack Web Development bootcamp · 2022 Ruby, Ruby on Rails, JavaScript, SQL/PostgreSQL, HTML/CSS.

Self-directed, 2022 – 2025 — LaunchSchool coursework, coding challenges and independent study alongside running the business, before moving back into engineering full time.